

Project: Production-Ready RAG Application —

Ask My Docs

Project Overview

Objective

Design and develop a **production-grade Retrieval-Augmented Generation (RAG) application** that enables users to upload their own documents and ask natural language questions. The application should retrieve the most relevant context from the uploaded documents using a hybrid retrieval pipeline and generate accurate, source-grounded responses with citations.

Unlike a basic chatbot, this system should focus on **retrieval quality, modular architecture, scalability, evaluation, and production deployment**.

Learning Outcomes

By completing this project, engineers will learn:

- End-to-end RAG architecture
- Document preprocessing pipeline
- Text chunking strategies
- Embedding generation
- Vector databases
- Hybrid Search (Semantic + Keyword)
- Cross Encoder Re-ranking
- Prompt Engineering for RAG
- FastAPI Backend Development
- React Frontend Development
- Docker Deployment

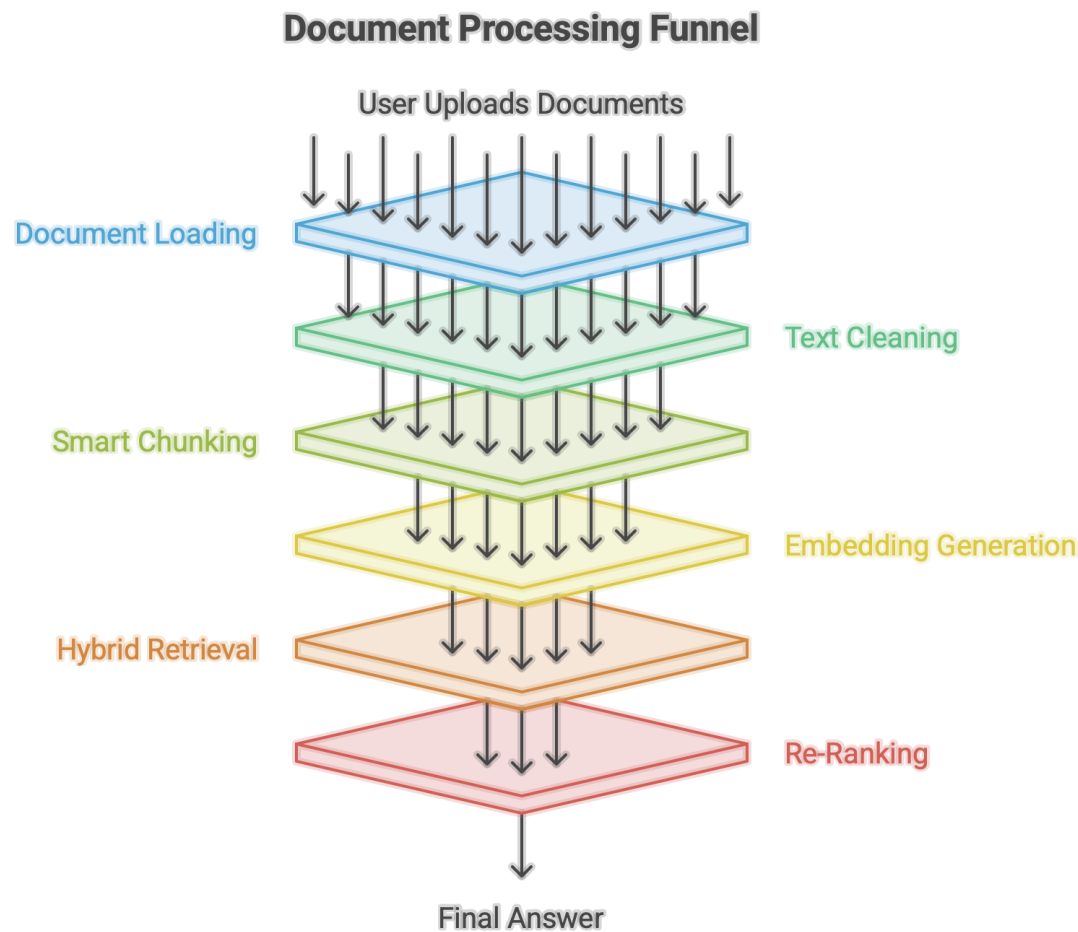
- RAG Evaluation using DeepEval/RAGAS
 - Production-ready software engineering practices
-

Tech Stack

Component	Technology
Backend	FastAPI
Frontend	React.js
LLM	Lllama 3.2 / Gemma 3
Embeddings	Sentence Transformers
Vector Database	ChromaDB
Keyword Retrieval	BM25
Re-ranking	Cross Encoder
Document Processing	PyMuPDF, python-docx
Evaluation	RAGAS / DeepEval

Deployment	Docker
Package Management	pip / uv
Version Control	Git & GitHub

System Architecture



Project Features

Core Features

- Upload multiple documents
 - PDF support
 - DOCX support
 - TXT support
 - Automatic text extraction
 - Metadata extraction
 - Intelligent chunking
 - Semantic embeddings
 - Hybrid retrieval
 - Cross Encoder re-ranking
 - Context-aware prompting
 - Source citations
 - Conversation history
 - FastAPI REST APIs
 - React dashboard
-

Advanced Features

- Multi-document querying
 - Chunk overlap optimization
 - Metadata filtering
 - Top-K configurable retrieval
 - Source highlighting
 - Confidence score
 - Query rewriting
 - Streaming responses
 - Conversation memory
-

Functional Modules

Module 1 — Document Upload

Goal

Allow users to upload documents securely.

Supported Formats

- PDF
- DOCX
- TXT

Responsibilities

- File validation
 - Duplicate detection
 - Size validation
 - Store uploaded files
-

Module 2 — Document Loader

Goal

Extract readable text.

Libraries

- PyMuPDF
- python-docx

Responsibilities

- Parse files
 - Extract text
 - Extract page numbers
 - Preserve document order
-

Module 3 — Cleaning Pipeline

Goal

Prepare text for embedding.

Tasks

- Remove extra spaces
- Remove unwanted characters
- Normalize Unicode
- Fix encoding
- Preserve paragraph boundaries
- Extract metadata

Metadata

- Filename
 - Page Number
 - Chunk ID
 - Upload Time
 - Document ID
-

Module 4 — Chunking

Goal

Create semantically meaningful chunks.

Recommended Strategy

- Recursive Character Text Splitter

Suggested Parameters

Chunk Size: 600–800 tokens

Chunk Overlap: 100–150 tokens

Store

- Chunk Text

- Metadata
 - Parent Document
-

Module 5 — Embedding Generation

Goal

Convert chunks into vectors.

Recommended Models

- all-MiniLM-L6-v2
- bge-small-en
- e5-base

Store

- Vector
 - Chunk
 - Metadata
-

Module 6 — Index Creation

Maintain two retrieval systems.

Vector Search

Using

- ChromaDB

Keyword Search

Using

- BM25

Both indexes should remain synchronized after uploads.

Module 7 — Hybrid Retrieval

Input: User Ques

Retrieve: Top Semantic Matches + Top Keyword Matches

Merge

Remove duplicates

Return

Top Candidates

Module 8 — Cross Encoder Re-ranking

Purpose

Improve retrieval quality.

Example Models

- ms-marco-MiniLM-L-6-v2

Input

Question

Candidate Chunks

Module 9 — Prompt Builder

Construct prompts dynamically.

Example Template:

You are an AI assistant.

Answer only using the provided context.

If the answer cannot be found,

reply that the information is unavailable.

Always cite the document and page number.

Context:

{Retrieved Chunks}

Question:

{User Question}

Module 10 — Response Generation

LLM

- Groq
- Gemini

Responsibilities

Generate

- Accurate answer
 - Grounded response
 - Structured response
 - Citations
-

Module 11 — Citation Engine

Each response should include:

- Answer
- Sources
- Document Name

- Page Number
- Chunk Reference

Example:

The refund policy allows cancellation within
30 days.

Sources

Employee Handbook.pdf

Page 14

HR Policy.pdf

Page 6

Module 12 — Evaluation

Evaluate using

RAGAS

Metrics

- Faithfulness
- Context Precision
- Context Recall
- Answer Relevancy

DeepEval

Evaluate

- Hallucination
 - Retrieval Quality
 - Answer Correctness
-

API Endpoints

Document APIs

POST /upload

GET /documents

DELETE /document/{id}

Chat APIs

POST /ask

POST /chat

GET /history

Health

GET /health

Project Structure

ask-my-docs/

|

|— backend/

| |— app/

| |

| |— api/

| | |— upload.py

| | |— chat.py

| | |— health.py

```
| |
| |─ core/
| |   │─ config.py
| |   │─ logger.py
| |   └─ security.py
| |
| |─ loaders/
| |   │─ pdf_loader.py
| |   │─ docx_loader.py
| |   └─ txt_loader.py
| |
| |─ preprocessing/
| |   │─ cleaner.py
| |   │─ metadata.py
| |   └─ chunker.py
| |
| |─ embeddings/
| |   └─ embedding_model.py
| |
| |─ vectorstore/
| |   │─ chroma_store.py
| |   └─ bm25_index.py
| |
| |─ retrieval/
```

- | | |└─ hybrid.py
- | | |└─ reranker.py
- | | └─ filters.py
- | |
- | └─ llm/
- | | └─ prompt_builder.py
- | | └─ generator.py
- | | └─ citation.py
- | |
- | └─ evaluation/
- | | └─ ragas_eval.py
- | | └─ deepeval_eval.py
- | |
- | └─ models/
- | └─ schemas/
- | └─ services/
- | └─ utils/
- | └─ main.py
- | └─ requirements.txt
- |
- └─ frontend/
- | └─ src/
- | | └─ pages/
- | | └─ components/

```
| | | └─ hooks/
| | | └─ services/
| | | └─ context/
| | └─ App.jsx
| └─ public/
| └─ package.json
|
└─ docker/
| └─ Dockerfile.backend
| └─ Dockerfile.frontend
| └─ docker-compose.yml
|
└─ data/
| └─ uploads/
| └─ chroma/
| └─ bm25/
|
└─ tests/
| └─ unit/
| └─ integration/
| └─ api/
|
└─ docs/
| └─ architecture.md
```

```
| |— api_docs.md
| |— deployment.md
|
|— .env
|— README.md
|— .gitignore
```

Engineering Guidelines

- Follow a modular architecture with clear separation of concerns.
 - Use type hints, Pydantic models, and consistent API validation.
 - Implement structured logging and centralized exception handling.
 - Write unit tests and integration tests for all critical modules.
 - Manage configuration through environment variables.
 - Store API keys securely; never commit secrets to version control.
 - Ensure code quality using formatting and linting tools.
 - Design retrieval and generation components to be independently replaceable.
-

Future Enhancements

- User authentication and role-based access control
- Multi-user document isolation
- Persistent chat sessions
- Streaming LLM responses (Server-Sent Events/WebSockets)
- OCR support for scanned PDFs and images
- Multilingual document support
- Advanced metadata filtering
- Query rewriting and decomposition
- Hybrid retrieval tuning with Reciprocal Rank Fusion (RRF)
- Response caching with Redis
- Monitoring and observability using Prometheus and Grafana
- Cloud deployment on AWS, Azure, or GCP
- CI/CD pipelines with GitHub Actions
- Rate limiting and API security
- Human feedback loop for continuous retrieval and prompt improvement

Expected Deliverables

At the end of this project, engineers should deliver:

1. A FastAPI backend implementing a modular RAG pipeline.
2. A React frontend for document management and conversational querying.
3. A hybrid retrieval system combining ChromaDB and BM25 with cross-encoder re-ranking.
4. Accurate, citation-based responses grounded in uploaded documents.
5. Automated evaluation reports using RAGAS and DeepEval.
6. Dockerized services with `docker-compose` for local deployment.
7. Comprehensive API documentation, architecture documentation, and setup instructions.
8. A production-ready GitHub repository following clean architecture and software engineering best practices.